DATA ENGINEER INTERVIEW PREP

Python • SQL • PySpark • AWS • ETL/ELT • Data Warehousing

Role-focused, anonymous preparation guide for the 18 Sep 2026 Data Engineer JD group

Main positioning

Hands-on AWS data engineering with Python, PySpark, SQL, Glue, S3, Lambda, Step Functions, Redshift, incremental processing, data validation, monitoring and APIs.

1. ROLE FIT

Target Role & Match Overview

- **Estimated ATS alignment: ~71%** for the combined JD group. The score is pulled down mainly by role seniority and platform-specific requirements, not by missing core data-engineering fundamentals.
- **Strong direct match:** Python, SQL/MySQL, PySpark, ETL/ELT, AWS, Glue, S3, Lambda, Step Functions, Redshift, Git/CI-CD, data pipelines, schema mapping, incremental processing, validation, monitoring and troubleshooting.
- **Partial / transferable match:** Apache Spark from PySpark experience; data warehousing through Redshift; orchestration through Step Functions; analytical storage and warehouse workflows; cloud-native pipeline operations.
- **Seniority mismatch:** several postings ask for 4+, 5-7, 6+, or 7+ years. Position the profile as a 3+ year hands-on engineer, not as a senior engineer.
- **Goal in interview:** prove strong fundamentals and production thinking, then explain how existing Spark/AWS concepts transfer to Databricks/Airflow/Snowflake/Azure.

High-Priority Keyword Map & Gaps

- **Priority 1 - already strong:** Python, SQL, PySpark, ETL/ELT, data pipelines, AWS, Glue, S3, Lambda, Redshift, MySQL, data validation, monitoring, Git and CI/CD.
- **Priority 2 - demonstrate with examples:** scalable ingestion, incremental loads, schema mapping, warehouse loading, data quality, orchestration, retries, idempotency, partitioning, error handling and observability.
- **Missing production tools:** Databricks, Delta Lake, Apache Airflow, Snowflake, DBT, BMC Control-M, Azure Data Factory and Azure Blob Storage.
- **Do not say:** “I worked on Databricks/Airflow/Snowflake” if you only know the concepts. Say exactly what you used and map the concepts.
- **Best gap bridge:** “I use PySpark on AWS Glue and Step Functions for orchestration. The underlying Spark, DAG, retry, partitioning, data-quality and warehouse concepts transfer directly; I am building targeted POCs for the platform-specific pieces.”

60-Second Introduction

- “I am a Data Engineer and Python Developer with 3+ years of experience. My recent work is focused on AWS data engineering, where I build ETL pipelines using Python, PySpark and AWS Glue, store and process data in S3, orchestrate workflows with Step Functions and Lambda, and use Redshift and DynamoDB for downstream processing.”
- “I have worked on finance data workflows involving invoice, remittance and credit-memo data. My responsibilities include schema mapping, incremental loads, data validation, API integration, monitoring in CloudWatch and production troubleshooting.”
- “Earlier I worked on Python backend applications with MySQL and REST APIs, so I am also comfortable with SQL optimization and service integration. I am now looking for a Data Engineer role where I can work on larger-scale pipelines and deepen my Spark and cloud-data-platform skills.”

Practice this until it sounds natural. Keep the first answer under about one minute; do not list every AWS service.

2-Minute Introduction

- **Start:** “I have 3+ years across software development and data engineering. My current focus is Python, SQL, PySpark and AWS-based ETL.”
- **Recent project:** explain the finance data pipeline: source data and APIs → S3 → Glue/PySpark transformation → Step Functions/Lambda orchestration → data validation → Redshift/downstream storage → CloudWatch alerts.
- **Second project:** explain the serverless document workflow using Lambda, SQS, S3, DynamoDB and Oracle ORDS APIs, with reprocessing and monitoring.
- **Earlier foundation:** Python REST APIs, MySQL, stored procedures, views, indexing and operational application support.
- **Close:** “The JD matches my core strengths in Python, SQL, Spark-based ETL and cloud pipelines. I do have platform gaps such as Databricks/Airflow/Snowflake, but I understand the core concepts and can transfer my existing experience quickly.”

Project Deep Dive: AWS Finance Data Pipeline

- **Problem:** process invoice, remittance and credit-memo data through a repeatable cloud data pipeline.
- **Scale:** approximately 30GB+ daily data in the verified project profile.
- **Flow:** S3 landing/data-lake layer → Glue/PySpark transformations → schema mapping, enrichment and data-quality checks → Step Functions/Lambda orchestration → Redshift/downstream storage.
- **Incremental strategy:** Glue bookmarks and incremental processing instead of reprocessing all historical data.
- **Reliability:** CloudWatch monitoring, SES/Teams alerts and explicit error handling; the verified profile reports a 40% downtime reduction.
- **Interview emphasis:** why you chose distributed processing, how you prevent duplicate loads, how you validate row counts/schema/nulls, and how you troubleshoot failed stages.

Project Deep Dive: Serverless Finance Automation

- **Problem:** replace a legacy finance-document process with a serverless AWS workflow.
- **Flow:** Oracle ORDS APIs provide business data; Lambda handles processing; SQS decouples work; S3 stores outputs; DynamoDB supplies enrichment/state; PDF/CSV templates generate final documents.
- **Data engineering angle:** API ingestion, validation, transformation, enrichment, deterministic naming, reprint/reprocessing logic and audit-friendly storage.
- **Reliability angle:** use queues and retry-safe processing so temporary failures do not lose messages or create duplicate outputs.
- **Verified impact:** the source profile reports an 80% reduction in manual document-processing effort.
- **Likely question:** “Why SQS?” Answer: decoupling, buffering traffic, retry behavior and protecting downstream services from spikes.

Project Deep Dive: Python + MySQL Platform

- **Use this project to prove SQL depth:** REST APIs, MySQL, reporting and workflow modules backed by operational data.
- **Database work:** stored procedures, views, functions, indexing and query improvements.
- **Integration work:** multiple internal systems and centralized operational data via APIs.
- **Good interview story:** when a query is slow, check execution plan, indexes, filters, join cardinality, data volume and whether computation can be pre-aggregated.
- **Bridge to data engineering:** backend experience helps with source-system APIs, transactional databases, schema contracts and downstream reliability.

ETL / ELT Fundamentals

What is ETL vs ELT?

ETL: extract, transform, then load; useful when transformation must happen before the target. **ELT:** extract and load raw data first, then transform inside a scalable warehouse/lakehouse. Choose based on platform, latency, governance and cost.

How would you design a batch pipeline?

Define source and SLA → land immutable raw data → validate schema and completeness → transform/standardize → deduplicate → load curated/warehouse layer → run quality checks → publish metrics and alerts → support retries/backfills.

What makes a pipeline production-ready?

Idempotency, observability, clear ownership, retry strategy, data-quality gates, schema handling, secrets/IAM, cost controls, backfill support and documented recovery procedures.

PySpark: Concepts You Must Explain

- **DataFrame:** distributed tabular abstraction with optimized execution; prefer built-in functions over Python UDFs where possible.
- **Lazy evaluation:** transformations build a logical plan; an action triggers execution.
- **Transformations vs actions:** select/filter/join/groupBy transform; count/collect/write trigger computation.
- **Partitioning:** controls parallelism and shuffle behavior. Too few partitions underuse the cluster; too many create scheduling/file overhead.
- **Wide vs narrow transformation:** wide operations such as joins/groupBy can trigger shuffles; narrow operations can often stay within a partition.
- **Cache/persist:** use only when a dataset is reused and recomputation is expensive; unpersist when finished.

PySpark Optimization & Troubleshooting

- **Start with evidence:** inspect Spark UI/job stages, skew, shuffle read/write, executor memory, spill and task duration.
- **Reduce data early:** select only needed columns and filter before expensive joins/aggregations.
- **Join strategy:** broadcast genuinely small dimension tables; avoid blindly broadcasting. Repartition on useful keys when needed.
- **Skew:** identify hot keys; consider salting, pre-aggregation, better partitioning, or separating exceptional keys.
- **Small files:** avoid excessive tiny outputs; control partition counts before write and use sensible file sizes.
- **Serialization/UDFs:** prefer Spark SQL/native functions; Python UDFs can add serialization overhead.
- **Interview line:** “I optimize based on the physical plan and runtime metrics, not by applying repartition/cache everywhere.”

SQL: Core Interview Patterns

- **Joins:** know inner, left, full and anti-join use cases; always think about duplicate keys and row multiplication.
- **Window functions:** ROW_NUMBER for dedup/latest record; SUM/AVG OVER for running metrics; LAG/LEAD for change detection.
- **Aggregation:** WHERE filters rows before GROUP BY; HAVING filters groups after aggregation.
- **Optimization:** index selective predicates/join keys, avoid SELECT *, inspect execution plans, and limit repeated scans.

Example: latest record per customer

```
SELECT customer_id, order_id, order_date
FROM (
SELECT customer_id, order_id, order_date,
ROW_NUMBER() OVER (PARTITION BY customer_id
ORDER BY order_date DESC) AS rn
FROM orders
) x
WHERE rn = 1;
```

Data Modeling & Warehousing

- **Fact table:** measurable business events, usually many rows - orders, payments, invoice lines.
- **Dimension table:** descriptive context - customer, product, date, geography.
- **Star schema:** simple and analytics-friendly; fact table directly connected to denormalized dimensions.
- **SCD Type 1:** overwrite history. **SCD Type 2:** preserve history with effective dates/version flags.
- **Warehouse vs lake:** warehouse is curated and query-optimized; data lake stores raw/structured/semi-structured data cheaply; modern lakehouse platforms combine aspects of both.
- **Your safe answer:** professional experience includes Redshift-oriented warehouse workflows and schema mapping; do not overclaim advanced enterprise dimensional-model ownership if not performed.

AWS Data Pipeline Architecture

- **S3:** durable raw/curated data-lake storage; organize by dataset/date/partition and lifecycle policy.
- **Glue + PySpark:** distributed transformation, schema mapping, data-quality logic and large-scale ETL.
- **Lambda:** lightweight event-driven control logic, API calls, metadata operations and glue code; not ideal for long heavy Spark transforms.
- **Step Functions:** workflow state machine for sequencing jobs, branching, retries, timeouts and failure paths.
- **SQS:** decoupling, buffering and asynchronous retries.
- **Redshift:** analytical warehouse layer; load curated data and design for query patterns.
- **CloudWatch/IAM:** logs, metrics, alarms and least-privilege access are part of the pipeline, not an afterthought.

Orchestration, Idempotency & Backfills

- **Orchestration:** coordinates dependencies and state; Step Functions is your production example. Airflow uses DAGs/scheduling/operators but solves similar dependency problems.
- **Idempotency:** re-running a job with the same input should not corrupt data or create duplicate results.
- **Practical methods:** deterministic keys, merge/upsert logic, processed-file metadata, checkpoint/bookmark state, partition overwrite or transactional write patterns.
- **Retries:** retry transient errors with bounded backoff; route permanent/business-data errors for investigation rather than infinite retry.
- **Backfill:** parameterize date/range, isolate historical runs, preserve lineage and avoid interfering with current production SLA.
- **Exactly-once caveat:** end-to-end exactly-once is difficult across external systems; design effectively-once behavior through idempotent sinks and deduplication.

Incremental Loads & Data Quality

- **Incremental choices:** timestamp watermark, source sequence/id, CDC log, partition discovery, Glue bookmarks, or snapshot diff depending on source capability.
- **Common checks:** schema/type validation, required fields, null thresholds, unique-key duplication, row-count/reconciliation, accepted ranges and referential integrity.
- **Quarantine pattern:** bad records should be isolated with reason codes instead of silently dropped.
- **Schema evolution:** detect additions/type changes, classify compatible vs breaking changes, version contracts and alert owners.
- **Late-arriving data:** use event timestamps and a correction window; design merges so old partitions can be safely updated.
- **Interview principle:** data-quality checks should be observable and actionable, not only boolean pass/fail.

Production Troubleshooting Scenarios

Glue job succeeds, but S3 output is empty. What do you check?

Input path/date partition, filters, row count after each transform, schema mismatch, job bookmark state, write path, IAM permission, logs and whether output was written to an unexpected partition.

Pipeline suddenly becomes 3x slower.

Compare data volume, skew and partition count; inspect Spark stages/shuffle; check source API/database latency; confirm cluster/executor resources; look for code/schema changes; verify downstream throttling.

Duplicate rows appear after retry.

Find the replay boundary, identify the business key, verify checkpoint/bookmark behavior and sink write semantics, then make the operation idempotent with merge/upsert/dedup or processed-input tracking.

A source schema changes in production.

Stop unsafe writes if needed, capture the new schema, classify compatibility, update mapping/tests, backfill affected data and add schema-change alerts/contracts to prevent silent corruption.

System Design: Scalable Daily Data Pipeline

- **Requirements:** define sources, daily volume, SLA, freshness, retention, consumers, recovery time and data-quality expectations.
- **Ingestion:** API/database/files → immutable S3 raw zone with source/date partitioning and metadata.
- **Processing:** Glue/PySpark reads only relevant partitions, validates schema, deduplicates, transforms and writes curated Parquet-style datasets/warehouse loads.
- **Orchestration:** Step Functions coordinates ingestion → quality gate → transform → warehouse load → publish/notify.
- **Reliability:** checkpoints/bookmarks, idempotent writes, DLQ/error bucket, retry rules and backfill parameters.
- **Observability:** job duration, input/output row counts, freshness, failures, data-quality metrics and cost; alerts to operations channels.
- **Security:** least-privilege IAM, encryption, secrets management, private networking where required and controlled access to sensitive datasets.

How to Answer the Platform Gaps

- **Databricks / Delta Lake:** “I have not used them in production. I use PySpark on AWS Glue, so Spark transformations, partitioning, joins, shuffles and optimization are familiar. I understand Delta adds transactional lakehouse features such as ACID tables, schema enforcement and merge/time-travel patterns.”
- **Airflow:** “My production orchestration is Step Functions. I understand Airflow DAGs, task dependencies, scheduling, retries and backfills; the orchestration concepts are transferable.”
- **Snowflake:** “My warehouse exposure is Redshift. I would map my SQL, loading, schema and warehouse concepts while learning Snowflake-specific virtual warehouses, stages and optimization.”
- **DBT:** “I have SQL transformation and validation experience but not production dbt. I understand it organizes SQL transformations as tested, documented models with dependency graphs.”
- **Azure / ADF / Blob:** “My cloud production experience is AWS. I can map S3→Blob, Glue/Step Functions concepts→ADF/Synapse patterns, and IAM concepts→Azure identity/RBAC, while learning the platform-specific services.”

Python / Data Engineering Coding Frameworks

- **File ingestion:** stream/read in chunks where needed, validate schema, normalize types, log bad records, write deterministic outputs.
- **Dedup:** define the true business key first; do not simply drop duplicates without understanding record precedence.
- **Error handling:** catch expected exceptions, add context, avoid swallowing errors, use retries only for transient failures.
- **Memory:** avoid loading huge datasets into local Python if distributed processing is available; use generators/chunks for local workloads.

Simple dedup pattern

```
seen = set()
result = []
for row in rows:
key = (row['customer_id'], row['event_id'])
if key not in seen:
seen.add(key)
result.append(row)
```

Behavioral / STAR Questions

- **Tell me about a production issue:** Situation → pipeline/data workflow problem; Task → restore reliability; Action → inspect logs/data/permissions, isolate root cause, fix, validate and add monitoring; Result → use only verified outcomes.
- **How do you prioritize:** prioritize production/SLA impact, unblock dependent work, communicate risk early, deliver in small verifiable steps.
- **Tell me about a disagreement:** focus on technical evidence, requirements and trade-offs; propose a small test/POC rather than arguing from preference.
- **How do you learn a new platform:** map known concepts first, build a small end-to-end POC, compare operational differences, then use it in a larger project.
- **Why job instead of freelancing:** emphasize stable long-term ownership, teamwork, larger systems and structured growth - not dissatisfaction with clients.
- **Why should we hire you:** strong hands-on Python/AWS/PySpark/ETL foundation, production troubleshooting mindset, and ability to contribute to cloud data pipelines while learning the platform-specific stack.

Final Revision Checklist

- **Must answer smoothly:** self-introduction, current pipeline architecture, ETL vs ELT, Spark lazy evaluation, partitions/shuffles, joins, SQL window functions, incremental load, idempotency, S3/Glue/Lambda/Step Functions/Redshift roles.
- **Prepare 3 stories:** pipeline build, performance/reliability improvement, production issue/troubleshooting.
- **Never overclaim:** Databricks, Airflow, Snowflake, dbt, Azure, Control-M or 5-7+ years.
- **Before interview:** revise one Spark optimization example, write 5 SQL queries by hand, draw your AWS pipeline once, and practice the 60-second introduction three times.
- **Questions to ask interviewer:** What is the daily data volume? Batch vs streaming? Primary warehouse/lakehouse? How are pipelines orchestrated and monitored? What are the biggest reliability or cost challenges?
- **Closing line:** “My strongest experience is Python, SQL, PySpark and AWS pipelines. I can contribute in those areas immediately while ramping quickly on your platform-specific tools.”

Interview strategy: Be concrete about what you built. When a tool is unfamiliar, explain the concept using the nearest production system you already know.